

COSC 81/281: Problem Set 0: Setup and Mathematical Preliminaries

Due Tuesday, September 22 (11:59pm ET)

Problem sets are graded **pass/fail on completion**, and there are no late days. The problem sets are where the learning happens, and they are the best available practice for the midterm and final exam. Submit to **Gradescope** (<https://www.gradescope.com/courses/1372528>) and assign each question to a page in your submission. Questions are welcome on **Slack**. Starred (★) questions are required for COSC 281 students and optional (ungraded) for COSC 81 students.

AI Policy. You may use any AI tools you like to assist you with your assignments, and you should, as that is likely how you will work for the rest of your career. That being said, the written portions are practice for the in-class midterm and exam, which are closed book and on paper. You should strive to answer these yourself and use AI as a tutor and not an oracle.

Please include an AI Tool Disclosure Below

Overview

This problem set has two goals. First, it reviews the mathematical foundations used throughout the course: linear algebra, basic probability, and derivatives. Second, it walks you through installing the course software environment and confirms that the simulator and testing tools run correctly on your machine. This is intentionally the lightest problem set of the term. If any single step, and in particular the software install, takes you more than an hour, please stop and ask for help on **Slack** rather than continuing to struggle alone.

Review resources. If a written problem below feels rusty, we encourage you to review the underlying material before grinding through the problem. The following free resources are matched to the parts of this assignment: 3Blue1Brown’s *Essence of Linear Algebra* and *Essence of Calculus* video series on YouTube (Parts 1 and 3), Khan Academy’s probability unit (Part 2), immersivemath.com for interactive linear algebra, and the free textbook *Mathematics for Machine Learning* (mml-book.github.io, Ch 2, 5, 6) as deeper reference. PS0 is half problem set and half tutorial: the released solutions state each rule before using it and link to further material.

What to submit. Submit to **Gradescope** (<https://www.gradescope.com/courses/1372528>):

1. **Written work (PDF):** your answers to Part 1 and the C1 trajectory image and observations. Space is provided below each problem; you may print this handout and write in it, annotate the PDF on a tablet, or use the provided `ps0/handout.tex`: enter each answer below its `%%% SOLUTION HERE %%%` marker, inside the corresponding `studentanswer` environment, and compile the file to produce your PDF. Assign each question to a page when you upload.
2. **Code:** your completed `warmup.py` from Part 2. Every test in this problem set is public, so if the autograder passes on your machine it will pass on submission.

Part 1: Written problems

Work each problem on paper, without a calculator; every answer comes out in small integers or simple fractions. Show your work in the space provided.

P1: Linear Algebra

a) Let $A = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$. Compute $A\mathbf{v}$.

b) The counterclockwise planar rotation matrix is $R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$. Evaluate $B = R(90^\circ)$. Let $A = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ be reflection across the x -axis and let $\mathbf{v} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$. Compute $AB\mathbf{v}$ (rotate first, then reflect) and $BA\mathbf{v}$ (reflect first, then rotate). Do the two results agree? Based on this example, is matrix multiplication commutative? Explain what this means about the order in which transformations are applied.

P2: Probability

- a) A discrete random variable X takes value 0 with probability $\frac{1}{2}$, value 2 with probability $\frac{1}{4}$, and value 4 with probability $\frac{1}{4}$. Compute $\mathbb{E}[X]$ and $\mathbb{E}[2X + 1]$.
- b) A bump sensor beeps with probability 0.9 when an obstacle is present and with probability 0.3 when none is. The prior probability of an obstacle is $\frac{1}{3}$. The sensor beeps once. Use **Bayes' rule** to compute the posterior probability that an obstacle is present.
- c) X takes value 0 with probability $\frac{1}{2}$ and value 2 with probability $\frac{1}{2}$. Compute $\mathbb{E}[X]$, $\mathbb{E}[X^2]$, and $\text{Var}[X] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$.
- d) Two range sensors fail independently, each with probability $\frac{1}{4}$. Compute the probability that both fail, and the probability that at least one fails. Which of the two computations *required* the independence assumption?

P3: Calculus

- a) For $f(x, y) = x^2 + 3xy$, compute the gradient $\nabla f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$ and evaluate it at $(1, 2)$. Then check each component numerically with the central-difference formula

$$\frac{\partial f}{\partial x_i}(\mathbf{x}) \approx \frac{f(\mathbf{x} + \epsilon \mathbf{e}_i) - f(\mathbf{x} - \epsilon \mathbf{e}_i)}{2\epsilon}$$

using $\epsilon = 0.1$. Compute both numerical estimates and state whether they agree with your analytical gradient.

- b) Use the chain rule: for $g(x) = (2x + 1)^2$, compute $g'(x)$ and evaluate $g'(1)$.

- c) Consider the loss function $L(w) = \frac{1}{2}(wx - y)^2$ with the single data point $x = 3$, $y = 5$. This is a standard squared-error loss in machine learning. Compute $\frac{dL}{dw}$, evaluate it at $w = 2$, and take one gradient descent step $w_1 = w_0 - \alpha \frac{dL}{dw}(w_0)$ with step size $\alpha = 0.1$.

P4: Deriving the Normal Equations ★ (required for 281; optional for 81)

The least squares algorithm can be derived directly from the two toolboxes this problem set reviews. Let A be an $m \times n$ matrix of inputs and $\mathbf{b}$ an m -vector of targets, and define $f(\mathbf{x}) = \|A\mathbf{x} - \mathbf{b}\|^2$.

a) Show that any minimizer satisfies the normal equations $A^\top A \mathbf{x} = A^\top \mathbf{b}$.

b) In one or two sentences, when is the solution unique?

Part 2: Programming

The programming part has three steps: install the course software environment, run a provided script to confirm the environment works, and implement five short functions that mirror the written problems above.

Installing the course environment

The course code requires **Python 3.12** and a terminal. The instructions below assume no prior experience with either; if you have set up Python projects before, the summary is: clone the repository, create a virtual environment at its root, and `pip install -r requirements.txt`.

Step 1: get the course code. The graphical option is GitHub Desktop: choose **File > Clone repository**, select <https://github.com/plancherb1/cosc-281-f26>, and choose where to keep the course folder. From a terminal, the equivalent commands are:

```
git clone git@github.com:plancherb1/cosc-281-f26.git
cd cosc-281-f26
```

Clone only once. When a new problem set is announced, use **Repository > Pull** in GitHub Desktop or run `git pull` from the course folder. A public fork is not required and should not contain your answers.

Step 2: open a terminal and check Python and Tk. Tk is the GUI component that opens the driving window; `pip` cannot install it.

- **macOS:** press Cmd-Space, type **Terminal**, and press return. Check your Python version with `python3 --version`. If the version is below 3.12 or the command is not found, install Python 3.12 from <https://www.python.org/downloads/> (use the macOS universal installer, then close and reopen the terminal). Run `python3 -m tkinter`; a small window should open. Close it after the check.
- **Windows:** install Python 3.12 from <https://www.python.org/downloads/>. On the first installer screen, **check the box labeled “Add python.exe to PATH”** before clicking Install; this is the step most people miss. Then open PowerShell (press the Windows key, type **PowerShell**, press enter) and check `python --version`. Note that on Windows the command is `python`, not `python3`. Then run `python -m tkinter`. If no small window opens, modify the Python installation and enable **Tcl/Tk and IDLE**.
- **Linux:** open your terminal application and check `python3 --version`. On Ubuntu or Debian, install what you need with `sudo apt install python3 python3-venv python3-tk`. Run `python3 -m tkinter` and close the small window after it appears.

Step 3: navigate to the course folder. If you used GitHub Desktop, open its **Repository** menu and choose **Open in Terminal**. You can also type `cd` followed by a space and the path to the cloned folder. On most systems you can type `cd` (with the trailing space) and then drag the folder from your file browser into the terminal window to fill in the path. Confirm you are in the right place: `ls` (macOS/Linux) or `dir` (Windows) should list `sim`, `ps0`, and `requirements.txt`.

Step 4: create and activate a virtual environment. A virtual environment is a private copy of Python's package installation for this course, so nothing here conflicts with anything else on your machine.

- **macOS / Linux:**

```
python3 -m venv --prompt COSC281 .venv
source .venv/bin/activate
```

- **Windows (PowerShell):**

```
python -m venv --prompt COSC281 .venv
.venv\Scripts\Activate.ps1
```

If PowerShell refuses to run the activation script with an error about execution policies, run `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser` once, answer yes, and try the activation again.

When activation succeeds, your terminal prompt begins with `(COSC281)`. This confirms that the course environment, rather than another project's environment, is active. You must activate it (the second command only) in every new terminal window you use for course work.

Step 5: install the course packages.

```
pip install -r requirements.txt
```

This downloads and installs the pinned course packages, including the course simulator (`coursesim`, which lives in `sim/`; the course scripts find it on their own, and there is nothing separate to install). It may take a few minutes.

Step 6: verify. Run the public test suite:

```
python ps0/code/autograder.py
```

The three environment tests should pass immediately. The ten `warmup` tests will fail with `NotImplementedError` until you complete C2 below; that is expected at this stage.

If anything above fails, copy the *entire* error message and post it on Slack along with your operating system. Installation problems are normal, they are fastest to fix with the full error text, and helping you through them is exactly what week one is for.

C1: Drive the car and plot the trajectory

Where to answer: your written-work PDF. If you use the editable TeX handout, place the image and observations inside the C1 `studentanswer` environment.

With the environment activated, run the teleoperation script:

```
python ps0/code/teleop_drive.py --map sim/maps/open.yaml \
    --out ps0/figures/ps0_teleop.png
```

A window opens showing a small car on an open map. Drive it with the arrow keys (or WASD) for at least 20 seconds of simulated time, then press `q` to quit. The script saves `ps0_teleop.png`, an image of the trajectory you drove drawn over the map. This script requires a display; if you work over ssh, use `ssh -X` or run locally. If no window opens, do not use a scripted trajectory as your answer; follow the installation fix printed by the command and retry until you can drive interactively.

Include the image in your PDF along with two or three sentences of observation: what motions could the car execute and what motions could it not (try to turn in place), and one thing about its behavior that you did not expect. The purpose of this task is to confirm that your environment works end to end and to give you direct experience with the car's steering constraint.

If you use the editable TeX handout, the image can be inserted at the C1 marker with:

```
\includegraphics[width=0.8\linewidth]{ps0/figures/ps0_teleop.png}
```

Write your two or three sentences below the image command.

C2: Five warmup functions

Where to answer: `warmup.py`. Write these implementations in the provided Python file and upload that file as the code submission. Do not paste the implementations into the written-work PDF.

The file `ps0/code/warmup.py` contains five function stubs. Each is the computational counterpart of a specific written problem. The docstring above each stub gives its contract, formula, and matching paper example. Read that contract before writing the function.

- `matvec(A, v)` implements P1(a). Use two explicit nested loops. You may use NumPy to convert the inputs, inspect their shapes, and allocate the output, but multiplication and accumulation must be scalar operations inside your loops. Do not use `@`, `np.dot`, `np.matmul`, `np.einsum`, or another routine that computes the product for you.
- `rot2d(theta)` constructs the rotation matrix from P1(b), with `theta` expressed in radians.
- `expectation(values, probs)` generalizes the sum in P2(a).
- `bayes_posterior(...)` generalizes the update in P2(b).
- `central_diff_grad(f, x, eps)` implements the numerical check in P3(a), one coordinate at a time.

Check your work by re-running the autograder:

```
python ps0/code/autograder.py
```

All thirteen tests pass when the environment and your five functions are correct. Every test in this problem set is public: there are no hidden tests, so a passing run on your machine is a passing run on Gradescope. Submit `warmup.py` (the only file you edit) along with your PDF.